

Auftrag: CODEE Coding Capability Hardening

Arbeite im Repository:

devdbzemusic/dbzs-codee-project

Ziel ist ausdrücklich **keine neue Architektur und kein großflächiger Umbau**. Der aktuelle Stand soll stabilisiert, verifiziert und zu einem belastbaren End-to-End Coding Assistant weiterentwickelt werden.

Hauptziel

CODEE muss einen realen Softwareentwicklungsauftrag vollständig und kontrolliert ausführen können:

```
Aufgabe verstehen
→ Projekt analysieren
→ relevante Dateien bestimmen
→ Änderung planen
→ Patch erstellen
→ Typecheck und Tests ausführen
→ Fehler auswerten
→ Patch korrigieren
→ finalen Diff präsentieren
→ Review Gate
→ optional Commit
```

Alle bestehenden Architekturentscheidungen sind zu respektieren. Insbesondere:

- Electron Main / Preload / Renderer Trennung
- FastAPI Backend
- Workspace Boundary
- Model Selection Broker
- Runtime Slots
- Job Spooler
- Review Gates
- ATIF-light Trajectory
- lokale Modelle zuerst
- kontrollierter Cloud-Fallback
- keine unkontrollierten Shell- oder Dateisystemzugriffe

Phase 1: Communication Spine verifizieren

Prüfe den aktuellen Stand auf `main`, insbesondere die Änderungen aus PR #20.

Untersuche und korrigiere mindestens folgende Punkte:

1. Non-Streaming Abort und Timeout

Verifiziere den vollständigen Pfad:

```
Renderer
→ backendClient
→ Preload Bridge
→ Electron Main IPC
→ AbortController
→ HTTP Request
→ FastAPI Backend
```

Anforderungen:

- Jede Non-Streaming-Anfrage erhält eine eindeutige `requestId`.
- `requestId` wird vollständig bis zum Electron Main Process übertragen.
- Electron Main verwaltet:

```
Map<requestId, AbortController>
```

- `cancelRuntimeChat(requestId)` muss den richtigen Request tatsächlich abbrechen.
- Abgebrochene Requests werden immer aus der Map entfernt.
- Cleanup erfolgt in `finally`.
- Stop-Button und Timeout müssen denselben kontrollierten Abbruchpfad verwenden.
- Ein abgebrochener Request darf keinen nachträglichen Assistant-Text mehr in den Store schreiben.

Erstelle dafür Unit- und Integrationstests.

2. Streaming Abort

Prüfe:

- Listener werden nur einmal registriert.
- Listener werden nach Abschluss entfernt.
- Abbruch beendet Stream, Reader und Backend-Verbindung.
- Keine verwaisten SSE- oder Fetch-Verbindungen.
- Kein doppeltes Schreiben in den RuntimeChatStore.

3. AbortSignal-Komposition

`AbortSignal.any()` darf nicht ohne Fallback verwendet werden.

Implementiere eine Utility, beispielsweise:

```
combineAbortSignals(signals: AbortSignal[]): {
  signal: AbortSignal;
```

```
cleanup: () => void;
}
```

Anforderungen:

- verwendet `AbortSignal.any`, wenn verfügbar
- besitzt einen kompatiblen Fallback
- entfernt registrierte Event Listener
- behandelt bereits abgebrochene Signale korrekt
- besitzt Tests

Phase 2: Model Selection Broker als echte Single Source of Truth

Ein Broker-Entscheid darf innerhalb desselben Runs nicht erneut berechnet oder überschrieben werden.

Prüfe den vollständigen Datenfluss:

```
runtimeChatStore
→ agentRunService
→ runtimeChatAgentRunner
→ backendClient
→ Electron Bridge
→ FastAPI Runtime API
→ RuntimeService
```

Der folgende Kontext muss vollständig weitergereicht werden:

```
{
  model_id: string | null;
  slot_id: string | null;
  provider: string;
  fallback_policy: string;
  routing_reason: string;
  decision_id: string;
}
```

Anforderungen:

- Entscheidung genau einmal pro User-Turn
- Agent Loop verwendet denselben Entscheid
- keine implizite Neuauswahl im Backend
- keine stille Ersetzung von `slot_id`
- keine globale Policy darf eine Request-Policy unbemerkt überschreiben

- Diagnostics zeigen exakt die tatsächlich verwendete Entscheidung
- Telemetry und Trajectory speichern dieselbe `decision_id`

Erstelle Tests für:

1. Strict Mode ohne Fallback
2. Local Fallback erlaubt
3. Cloud Fallback erlaubt
4. Slot nicht bereit
5. Modell nicht gefunden
6. Runtime während des Runs beendet
7. Agent Loop mit mehreren Schritten
8. Retry nach Transportfehler

Phase 3: Fallback Policy korrigieren

Prüfe insbesondere `RuntimeService.resolve_chat_target()` und alle angrenzenden Methoden.

Die Request-Policy hat Vorrang vor einer globalen Default-Policy.

Erwartetes Prinzip:

```
effective_policy = request.fallback_policy or self.default_fallback_policy
```

Die Policy muss explizit typisiert und validiert werden.

Erlaubte Werte sollen zentral definiert werden, beispielsweise:

```
strict
allow_local_fallback
allow_cloud_fallback
allow_any_fallback
```

Keine String-Verteilung über mehrere Dateien.

Erstelle eine zentrale Enum beziehungsweise einen Shared Contract.

Phase 4: End-to-End Coding Capability Test Harness

Erstelle innerhalb des Repositories ein kleines kontrolliertes Testprojekt:

```
test-fixtures/coding-capability-project/
```

Das Projekt soll absichtlich mehrere Fehler enthalten:

- TypeScript-Typfehler
- fehlerhafter Import
- Logikfehler
- fehlender Testfall
- fehlerhafte Konfiguration
- eine Datei, die nicht verändert werden darf
- eine Aufgabe, die mehrere Dateien betrifft

Beispielauftrag:

```
Repariere die Benutzerregistrierung.  
Die E-Mail-Validierung ist fehlerhaft, ein Import fehlt,  
die Tests decken doppelte Benutzer nicht ab und die öffentliche API  
darf nicht verändert werden.
```

CODEE muss dafür:

1. Projekt inventarisieren
2. relevante Dateien auswählen
3. Plan erstellen
4. Patch erzeugen
5. Tests ausführen
6. Fehler erkennen
7. Patch korrigieren
8. finalen Diff erzeugen
9. Review Gate anfordern
10. keine geschützte Datei verändern

Erstelle einen automatisierten Acceptance Test.

Mögliches Kommando:

```
pnpm test:coding-loop
```

Der Test muss reproduzierbar und offline ausführbar sein.

Phase 5: Repository Map und Kontextselektion

Implementiere keine vollständige neue RAG-Plattform.

Baue zunächst eine deterministische Repository Map mit:

- Dateibaum
- Sprachen
- Imports
- Exports
- Klassen
- Funktionen
- Tests
- Konfigurationsdateien
- Entry Points
- Git Status
- Änderungsabhängigkeiten

Die Map soll kompakt und modellfreundlich sein.

Beispiel:

```
{
  "path": "src/services/userService.ts",
  "language": "typescript",
  "symbols": [
    "createUser",
    "validateEmail"
  ],
  "imports": [
    "../userRepository"
  ],
  "referencedBy": [
    "src/api/register.ts",
    "tests/userService.test.ts"
  ]
}
```

Anforderungen:

- inkrementell aktualisierbar
- Workspace Boundary beachten
- keine Binärdateien
- keine Secrets
- keine `node_modules`
- keine Build-Artefakte
- Tokenbudget berücksichtigen

Phase 6: Sicherer Patch- und Repair-Loop

Der autonome Repair Loop muss harte Grenzen besitzen.

Pflichtfelder:

```
maxIterations
maxFilesChanged
maxPatchSize
maxCommandCount
timeoutMs
requireReviewGate
allowCommit
```

Standardwerte konservativ wählen.

Vor jedem Schreibvorgang:

- aktuellen SHA oder Dateinhalt speichern
- Restore Point erzeugen
- geplante Änderung protokollieren

Nach jedem Patch:

- Formatierung
- Typecheck
- gezielte Tests
- bei Bedarf vollständige Tests

Kein Commit ohne erfolgreiches Review Gate.

Phase 7: Dokumentation bereinigen

Reduziere widersprüchliche Statusdokumente.

Verbindliche Statusquellen:

```
README.md
docs/STATUS_MATRIX.md
docs/ROADMAP.md
HANDOVER.md
```

Alte Completion Reports verschieben nach:

```
docs/archive/
```

`STATUS_MATRIX.md` soll ausschließlich folgende Zustände verwenden:

REAL
PARTIAL
MOCK
BLOCKED
DEPRECATED

Keine Behauptung wie „Production Ready“, wenn die vollständigen Acceptance Tests nicht tatsächlich ausgeführt wurden.

Jeder Status benötigt:

- Implementierungsnachweis
- Testnachweis
- bekannte Einschränkungen
- verantwortliches Modul

Qualitätsanforderungen

Vor Abschluss müssen erfolgreich laufen:

```
pnpm typecheck
pnpm test
pnpm build
pnpm doctor:all
pnpm smoke-test
pnpm test:capabilities
pnpm test:coding-loop
```

Falls ein bestehender Test fehlschlägt:

- nicht deaktivieren
- nicht überspringen
- Ursache analysieren
- sauber beheben

Keine Änderungen an Tests ausschließlich mit dem Ziel, einen fehlerhaften Codepfad grün erscheinen zu lassen.

Arbeitsweise

1. Zuerst Audit des aktuellen `main`
2. Befunde nach P0, P1 und P2 klassifizieren
3. kleinen Implementierungsplan erstellen
4. Änderungen in logisch getrennten Commits umsetzen

5. Tests nach jedem Block ausführen
6. am Ende vollständigen Smoke-Test durchführen
7. Pull Request erstellen

Empfohlene Commits:

```
fix(runtime): implement real non-stream request cancellation
fix(routing): enforce request-scoped fallback policy
fix(chat): add abort signal compatibility utility
test(capability): add deterministic coding-loop fixture
feat(context): add repository map foundation
feat(agent): harden bounded repair loop
docs(status): consolidate project status sources
```

Definition of Done

Die Aufgabe ist erst abgeschlossen, wenn:

- Non-Streaming-Abbruch nachweislich den echten Request beendet
- Streaming-Abbruch keine Verbindung zurücklässt
- Broker-Entscheid durch alle Agentenschritte identisch bleibt
- Fallback Policy request-spezifisch funktioniert
- ein reales Testprojekt automatisch repariert wird
- geschützte Dateien unverändert bleiben
- Tests und Build vollständig grün sind
- Statusdokumentation dem tatsächlichen Stand entspricht
- keine offenen P0- oder P1-Review-Befunde verbleiben

Am Ende liefere:

1. Audit-Befunde
2. geänderte Dateien
3. Architekturentscheidungen
4. Testresultate
5. bekannte Restprobleme
6. nächste sinnvolle Phase
7. Link zum Pull Request